Manual de Usuario - Sistema de Torneos y Apuestas TournaBet

Backend API v1.0

1 de marzo de 2026

Índice

1. Introducción **3**
- 1.1. £Qué es TournaBet? . . . 3
- 1.2. Novedades de la Versión 2.0 . . . 3
- 1.3. Usuario Administrador Inicial . . . 3

2. Arquitectura del Sistema **4**
- 2.1. Modelo de Datos . . . 4
- 2.2. Tipos de Torneos Soportados . . . 4
- 2.3. Tipos de Selección Disponibles . . . 5

3. Flujo de Uso - Paso a Paso **6**
- 3.1. Paso 1: Crear una Categoría . . . 6
- 3.2. Paso 2: Crear un Torneo . . . 6
- 3.3. Paso 3: Crear Sesiones (Jornadas) . . . 7
- 3.4. Paso 4: Crear Eventos Globales (NUEVO) . . . 7
- 3.5. Paso 5: Asignar Evento a un Torneo . . . 8
- 3.6. Paso 6: Establecer Competidores . . . 8
- 3.7. Paso 7: Crear Selecciones (Opciones de Apuesta) . . . 9
- 3.8. Paso 8: Usuario se Registra e Inscribe . . . 10
- 3.9. Paso 9: Usuario hace Pronósticos . . . 10
- 3.10. Paso 10: Liquidar el Evento . . . 11
- 3.11. Paso 11: Consultar Leaderboard . . . 11

4. Ejemplo Completo - Tournament de Caballos **12**
- 4.1. 1. Crear Categoría . . . 12
- 4.2. 2. Crear Torneo . . . 12
- 4.3. 3. Crear Sesiones (5 jornadas) . . . 12
- 4.4. 4. Crear Eventos Globales . . . 12
- 4.5. 5. Asignar Carreras al Torneo . . . 13
- 4.6. 6. Establecer Competidores para cada Carrera . . . 13
- 4.7. 7. Crear Selecciones para cada Evento . . . 13
- 4.8. 8. Liquidar Resultados . . . 13

5. Referencia de API Completa **15**
5.1. Autenticación 15
5.2. Categorías 15
5.3. Torneos 15
5.4. Sesiones 15
5.5. Eventos Globales (NUEVO) 16
5.6. Asignación de Eventos a Torneos (NUEVO) 16
5.7. Selecciones 16
5.8. Competidores (Catálogo Global) 16
5.9. Billetera y Pagos 16
5.10. Usuarios (Admin) 17
5.11. Settlement (Liquidaciones) 17
5.12. Formatos de Respuesta 17

6. Códigos de Estado HTTP **19**
6.1. Cabeceras de Autenticación 19
6.2. Rate Limiting 19

7. Códigos de Error Comunes **20**
7.1. Errores de Autenticación 20
7.2. Errores de Torneos 20
7.3. Errores de Eventos 20

8. Apéndice: Ejemplos de Integración **21**
8.1. Ejemplo: Integración Frontend - Obtener Torneos 21
8.2. Ejemplo: Integración Frontend - Hacer Picks 21
8.3. Ejemplo: Webhook de Liquidación (futuro) 21

1. Introducción

Este documento es una guía práctica para usar el Sistema de Torneos y Apuestas **TournaBet**. El sistema permite gestionar torneos deportivos donde los participantes realizan selecciones y acumulan puntos según los resultados.

1.1. £Qué es TournaBet?

Es una plataforma para crear torneos de pronósticos deportivos (como caballos, fútbol, beisbol, etc.) donde:

- **Admin**: Crea torneos, categorías, sesiones, eventos globales y los asigna a torneos
- **Usuario**: Se registra, participa, hace pronósticos y acumula puntos en el leaderboard

1.2. Novedades de la Versión 2.0

Esta versión introduce importantes mejoras:

- **Eventos Globales**: Los eventos se crean una vez y pueden reutilizarse en múltiples torneos
- **Tabla TournamentEvents**: Nueva tabla intermedia para vincular eventos a torneos
- **Puntos por Tipo de Selección**: Configuración personalizada de puntos ganados por cada tipo de a
- **SuperLine**: Línea especial para sesiones de playoffs o finales
- **Catálogo Global de Competidores**: Competidores pueden tener odds, RL y SRL específicos por evento

1.3. Usuario Administrador Inicial

Al ejecutar la aplicación por primera vez, se crea automáticamente:

- **Email**: admin@betsystem.com
- **Password**: Admin123!

2. Arquitectura del Sistema

2.1. Modelo de Datos

```
Tournament (Torneo)
  |
  +-- Category (Categoría)
  +-- Session (Sesión/Jornada)
  |     +-- SuperLine (Línea especial para playoffs)
  |
  +-- TournamentEvent (Tabla Intermedia)
        |
        +-- Event (Evento Global)
              +-- Venue (Lugar)
              +-- Line (Línea)
              +-- TotalScore (Score total)
              |
              +-- PickableSelection (Opciones de apuesta)
              |     +-- SelectionType (Tipo de selección)
              |
              +-- Competitor (Competidor)
                    +-- Odds, RL, SRL (Cuotas específicas por evento)

Participant (Participante)
  |
  +-- UserPick (Selección del usuario)
        +-- PickableSelection (Opción apostada)
```

2.2. Tipos de Torneos Soportados

- **Fútbol**: Apuestas tradicionales (ganador, línea de dinero, over/under)
- **Beisbol**: Runline, over/under
- **Caballos**: Macho/Hembra, Alta/Baja

2.3. Tipos de Selección Disponibles

Tipo	Código	Descripción
Macho	macho	Apuesta al caballo/equipo favorito o seleccionado
Hembra	hembra	Apuesta al caballo/equipo no favorito
Alta	alta	Apuesta a que el score total será mayor a la línea
Baja	baja	Apuesta a que el score total será menor a la línea
Runline	runline	Línea de carrera (diferencial de puntos)
Super Alta	sralta	Sobre alta con línea especial (SuperLine)
Super Baja	srbaja	Sobre baja con línea especial (SuperLine)

3. Flujo de Uso - Paso a Paso

3.1. Paso 1: Crear una Categoría

Las categorías organizan los torneos (Fútbol, Caballos, Beisbol, etc.):

```
POST /api/v1/admin/categories
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "name": "Fútbol",
  "description": "Torneos de fútbol soccer",
  "icon": "soccer-ball"
}
```

3.2. Paso 2: Crear un Torneo

El admin crea un torneo con configuración de puntos:

```
POST /api/v1/admin/tournaments
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "name": "Copa Mundial 2026",
  "category_id": 1,
  "description": "Pronósticos del mundial",
  "start_date": "2026-06-01T00:00:00Z",
  "end_date": "2026-07-15T23:59:59Z",
  "entry_fee": 500,
  "max_participants": 100,
  "settings": {
    "selections_per_session": 3,
    "total_sessions": 8,
    "required_selection_types": ["macho", "alta", "baja"],
    "points_per_selection_type": {
      "macho": 10,
      "hembra": 15,
      "alta": 8,
      "baja": 8,
      "runline": 12,
      "sralta": 20,
      "srbaja": 20
    }
  }
}
```

Campos importantes:

- **max_participants**: Límite máximo de usuarios que pueden inscribirse (0 = ilimitado)

- **points_per_selection_type**: Puntos awarded por cada tipo de selección acertada
- **required_selection_types**: Tipos de selecciones obligatorias por sesión

3.3. Paso 3: Crear Sesiones (Jornadas)

Cada torneo tiene varias sesiones. Ej: 8 jornadas para un mundial:

```
POST /api/v1/admin/sessions
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "tournament_id": 1,
  "session_number": 1,
  "start_time": "2026-06-01T10:00:00Z",
  "end_time": "2026-06-01T14:00:00Z",
  "super_line": null,
  "description": "Jornada 1 - Fase de grupos"
}
```

Campo especial - SuperLine:

- Para sesiones normales de playoffs o finales, usar **super_line** para establecer una línea especial
- Los tipos **sralta** y **srbaja** usarán esta línea en lugar de la línea regular

3.4. Paso 4: Crear Eventos Globales (NUEVO)

Los eventos se crean una vez y pueden reutilizarse en múltiples torneos:

```
POST /api/v1/admin/events
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "name": "España vs Francia",
  "venue": "Santiago Bernabéu",
  "line": 2.5,
  "total_score": 0,
  "start_time": "2026-06-15T20:00:00Z",
  "status": "scheduled"
}
```

Campos del Evento:

- **name**: Nombre del evento (ej: .España vs Francia", Çarrera 5")
- **venue**: Lugar del evento (estadio, hipódromo, etc.)
- **line**: Línea base para apuestas Alta/Baja
- **total_score**: Score total actual (se actualiza al liquidar)

- **start_time**: Hora de inicio del evento
- **status**: Estado (scheduled, live, settled, cancelled)

3.5. Paso 5: Asignar Evento a un Torneo

Vincular un evento global a un torneo específico:

```
POST /api/v1/admin/tournament-events
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "event_id": 1,
  "tournament_id": 1,
  "session_id": 1,
  "order": 1
}
```

O alternativamente, ver eventos disponibles para asignar:

```
GET /api/v1/admin/events/available?tournament_id=1
Authorization: Bearer <token_admin>
```

3.6. Paso 6: Establecer Competidores

Para cada evento, definir los competidores (equipos, caballos, etc.):

```
POST /api/v1/admin/events/{id}/competitors
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "competitors": [
    {
      "name": "Selección Española",
      "assigned_number": 1,
      "odds": 1.85,
      "rl": -1.5,
      "srl": -2.5
    },
    {
      "name": "Selección Francesa",
      "assigned_number": 2,
      "odds": 2.10,
      "rl": 1.5,
      "srl": 2.5
    }
  ]
}
```

Campos del Competidor por Evento:

- **name**: Nombre del competidor

- **assigned_number**: Número asignado (1 para local, 2 para visitante)
- **odds**: Cuota decimal para apuesta de línea de dinero
- **rl**: Runline (para beisbol)
- **srl**: Super Runline (para sesiones con SuperLine)

3.7. Paso 7: Crear Selecciones (Opciones de Apuesta)

Las selecciones son las opciones en las que los usuarios pueden apostar:

```
POST /api/v1/admin/events/selections
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "event_id": 1,
  "description": "España gana",
  "selection_type": "macho",
  "competitor_id": 1,
  "line": null,
  "points_for_win": 10
}
```

```
POST /api/v1/admin/events/selections
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "event_id": 1,
  "description": "Más de 2.5 goles",
  "selection_type": "alta",
  "competitor_id": null,
  "line": 2.5,
  "points_for_win": 8
}
```

```
POST /api/v1/admin/events/selections
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "event_id": 1,
  "description": "Menos de 2.5 goles",
  "selection_type": "baja",
  "competitor_id": null,
  "line": 2.5,
  "points_for_win": 8
}
```

Reglas:

- Para tipos **macho** y **hembra**: Se debe especificar **competitor_id**

- Para tipos **alta**, **baja**, **sralta**, **srbaja**: Se debe especificar **line**
- **points_for_win**: Puntos awarded por esta selección (sobrescribe la configuración del torneo)

3.8. Paso 8: Usuario se Registra e Inscribe

El usuario se registra, verifica su email, recarga su billetera y se inscribe:

```
# Registro
POST /api/v1/auth/register
Content-Type: application/json

{
  "email": "usuario@email.com",
  "password": "Password123!",
  "name": "Juan Pérez",
  "username": "juanperez"
}
```

```
# Login
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "usuario@email.com",
  "password": "Password123!"
}
```

```
# Recargar billetera
POST /api/v1/wallet/deposit
Content-Type: application/json
Authorization: Bearer <token_usuario>

{
  "amount": 1000,
  "payment_method_id": 1
}
```

```
# Inscribirse al torneo
POST /api/v1/tournaments/1/join
Authorization: Bearer <token_usuario>
```

3.9. Paso 9: Usuario hace Pronósticos

El usuario hace sus selecciones para una sesión específica:

```
POST /api/v1/tournaments/1/sessions/picks
Content-Type: application/json
Authorization: Bearer <token_usuario>

{
```

```
  "session_id": 1,
  "selection_ids": [1, 2, 3]
}
```

Validaciones:

- El número de selecciones debe coincidir con **selections_per_session** del torneo
- Los tipos de selecciones deben incluir los **required_selection_types**
- Las selecciones deben corresponder a eventos de la sesión especificada
- El usuario no puede hacer pronósticos después del **end_time** de la sesión

3.10. Paso 10: Liquidar el Evento

Cuando termina el evento, el admin establece los resultados:

```
POST /api/v1/admin/events/{id}/settle
Content-Type: application/json
Authorization: Bearer <token_admin>

{
  "results": [
    {"competitor_id": 1, "final_score": 2, "position": null},
    {"competitor_id": 2, "final_score": 1, "position": null}
  ]
}
```

Para carreras de caballos:

```
{
  "results": [
    {"competitor_id": 1, "final_score": null, "position": 1},
    {"competitor_id": 2, "final_score": null, "position": 2},
    {"competitor_id": 3, "final_score": null, "position": 3},
    {"competitor_id": 4, "final_score": null, "position": 4}
  ]
}
```

El sistema automáticamente:

1. Actualiza el **total_score** del evento
2. Evalúa cada selección (alta/baja/macho/hembra/etc.)
3. Marca las selecciones ganadoras/perdedoras
4. Suma los puntos configurados a los usuarios que acertaron
5. Actualiza el leaderboard del torneo

3.11. Paso 11: Consultar Leaderboard

Ver el ranking de participantes:

```
GET /api/v1/tournaments/1/leaderboard
```

```
GET /api/v1/tournaments/1/leaderboard?session_id=1
```

4. Ejemplo Completo - Tournament de Caballos

4.1. 1. Crear Categoría

```
{
  "name": "Caballos",
  "description": "Carreras de caballos",
  "icon": "horse"
}
```

4.2. 2. Crear Torneo

```
{
  "name": "Gran Premio Hípico 2026",
  "category_id": 1,
  "description": "Torneo de 5 carreras",
  "start_date": "2026-03-01T00:00:00Z",
  "end_date": "2026-03-05T23:59:59Z",
  "entry_fee": 100,
  "max_participants": 50,
  "settings": {
    "selections_per_session": 4,
    "total_sessions": 5,
    "required_selection_types": ["macho", "hembra", "alta", "baja
   "],
    "points_per_selection_type": {
      "macho": 10,
      "hembra": 15,
      "alta": 8,
      "baja": 8
    }
  }
}
```

4.3. 3. Crear Sesiones (5 jornadas)

```
{
  "tournament_id": 1,
  "session_number": 1,
  "start_time": "2026-03-01T10:00:00Z",
  "end_time": "2026-03-01T14:00:00Z",
  "description": "Jornada 1 - 5 Carreras"
}
```

4.4. 4. Crear Eventos Globales

Crear las 5 carreras como eventos globales:

```
{
  "name": "Carrera 1",
  "venue": "Hipódromo La Rinconada",
  "line": 180,
  "total_score": 0,
  "start_time": "2026-03-01T15:00:00Z"
}
```

4.5. 5. Asignar Carreras al Torneo

```
{
  "event_id": 1,
  "tournament_id": 1,
  "session_id": 1,
  "order": 1
}
```

(Repetir para las 5 carreras)

4.6. 6. Establecer Competidores para cada Carrera

```
{
  "competitors": [
    {"name": "El Trueno", "assigned_number": 1, "odds": 2.5},
    {"name": "La Flecha", "assigned_number": 2, "odds": 3.0},
    {"name": "Velocista", "assigned_number": 3, "odds": 4.0},
    {"name": "Estrellita", "assigned_number": 4, "odds": 5.0}
  ]
}
```

4.7. 7. Crear Selecciones para cada Evento

Para cada carrera, crear las 4 selecciones requeridas:

- Macho (apostar a un caballo específico)
- Hembra (apostar a otro caballo)
- Alta (tiempo total mayor a línea)
- Baja (tiempo total menor a línea)

4.8. 8. Liquidar Resultados

Cuando termina la carrera:

```
{
  "results": [
    {"competitor_id": 1, "position": 1, "final_time": 120.5},
    {"competitor_id": 2, "position": 2, "final_time": 121.0},
    {"competitor_id": 3, "position": 3, "final_time": 122.0},
```

```
    {"competitor_id": 4, "position": 4, "final_time": 123.0}
  ]
}
```

El sistema determina automáticamente qué selecciones fueron ganadoras según:

- **Macho**: El competidor con position=1 fue el seleccionado
- **Hembra**: El resto de competidores
- **Alta/Baja**: Según el tiempo total vs la línea

5. Referencia de API Completa

5.1. Autenticación

Método	Endpoint	Descripción
POST	/api/v1/auth/register	Registrar nuevo usuario
POST	/api/v1/auth/login	Iniciar sesión (retorna JWT)
POST	/api/v1/auth/refresh	Renovar token JWT
GET	/api/v1/auth/me	Obtener usuario actual

5.2. Categorías

Método	Endpoint	Descripción
GET	/api/v1/categories	Listar categorías (público)
GET	/api/v1/categories/:id	Ver categoría
POST	/api/v1/admin/categories	Crear categoría (admin)
PUT	/api/v1/admin/categories/:id	Actualizar categoría (admin)
DELETE	/api/v1/admin/categories/:id	Eliminar categoría (admin)

5.3. Torneos

Método	Endpoint	Descripción
GET	/api/v1/tournaments	Listar torneos disponibles
GET	/api/v1/tournaments/:id	Ver detalle de torneo
GET	/api/v1/tournaments/:id/sessions	Ver sesiones del torneo
GET	/api/v1/tournaments/:id/events	Ver eventos del torneo
GET	/api/v1/tournaments/:id/leaderboard	Ver ranking
POST	/api/v1/tournaments/:id/join	Inscribirse al torneo
GET	/api/v1/tournaments/:id/mypicks	Ver mis selecciones
POST	/api/v1/tournaments/:id/sessions/picks	Hacer pronósticos
POST	/api/v1/admin/tournaments	Crear torneo (admin)
PUT	/api/v1/admin/tournaments/:id	Actualizar torneo (admin)
DELETE	/api/v1/admin/tournaments/:id	Eliminar torneo (admin)

5.4. Sesiones

Método	Endpoint	Descripción
GET	/api/v1/admin/sessions	Listar sesiones (admin)
POST	/api/v1/admin/sessions	Crear sesión (admin)
PUT	/api/v1/admin/sessions/:id	Actualizar sesión (admin)
DELETE	/api/v1/admin/sessions/:id	Eliminar sesión (admin)

5.5. Eventos Globales (NUEVO)

Método	Endpoint	Descripción
GET	/api/v1/events	Listar eventos globales (público)
GET	/api/v1/events/:id	Ver evento global
GET	/api/v1/events/tournament/:id	Ver eventos de un torneo
POST	/api/v1/admin/events	Crear evento global (admin)
PUT	/api/v1/admin/events/:id	Actualizar evento (admin)
DELETE	/api/v1/admin/events/:id	Eliminar evento (admin)
POST	/api/v1/admin/events/:id/competitors	Agregar competidores (admin)
DELETE	/api/v1/admin/events/:id/competitors/:cid	Eliminar competidor (admin)
POST	/api/v1/admin/events/:id/settle	Liquidar evento (admin)
GET	/api/v1/admin/events/available	Eventos disponibles para asignar

5.6. Asignación de Eventos a Torneos (NUEVO)

Método	Endpoint	Descripción
POST	/api/v1/admin/tournament-events	Asignar evento a torneo
DELETE	/api/v1/admin/tournament-events/:id	Quitar evento del torneo
GET	/api/v1/tournaments/:id/events	Ver eventos del torneo

5.7. Selecciones

Método	Endpoint	Descripción
GET	/api/v1/events/:id/selections	Ver selecciones de evento
POST	/api/v1/admin/events/selections	Crear selección (admin)
PUT	/api/v1/admin/events/selections/:id	Actualizar selección (admin)
DELETE	/api/v1/admin/events/selections/:id	Eliminar selección (admin)

5.8. Competidores (Catálogo Global)

Método	Endpoint	Descripción
GET	/api/v1/competitors	Listar competidores globales
GET	/api/v1/competitors/:id	Ver competidor
POST	/api/v1/admin/competitors	Crear competidor (admin)
PUT	/api/v1/admin/competitors/:id	Actualizar competidor (admin)
DELETE	/api/v1/admin/competitors/:id	Eliminar competidor (admin)
GET	/api/v1/competitors/search	Buscar competidores por nombre

5.9. Billetera y Pagos

Método	Endpoint	Descripción
GET	/api/v1/wallet/balance	Ver saldo
POST	/api/v1/wallet/deposit	Depositar fondos
POST	/api/v1/wallet/withdraw	Retirar fondos
GET	/api/v1/wallet/transactions	Historial de transacciones
GET	/api/v1/payment-methods	Métodos de pago disponibles
POST	/api/v1/admin/payment-methods	Crear método de pago (admin)

5.10. Usuarios (Admin)

Método	Endpoint	Descripción
GET	/api/v1/admin/users	Listar usuarios (admin)
GET	/api/v1/admin/users/:id	Ver usuario (admin)
PUT	/api/v1/admin/users/:id	Actualizar usuario (admin)
DELETE	/api/v1/admin/users/:id	Eliminar usuario (admin)
GET	/api/v1/admin/tournament-participants/:id	Participantes de un torneo
PUT	/api/v1/admin/tournament-participants/:id/status	Cambiar estado de participante

5.11. Settlement (Liquidaciones)

Método	Endpoint	Descripción
GET	/api/v1/admin/settlements	Listar liquidaciones (admin)
GET	/api/v1/admin/settlements/:id	Ver liquidación (admin)
POST	/api/v1/admin/events/:id/settle	Liquidar evento (admin)
GET	/api/v1/tournaments/:id/my-settlements	Mis resultados (usuario)

5.12. Formatos de Respuesta

Éxito - GET:

```
{
  "success": true,
  "data": {
    "id": 1,
    "name": "Ejemplo",
    ...
  }
}
```

Éxito - POST/PUT:

```
{
  "success": true,
  "message": "Operación exitosa",
  "data": { ... }
}
```

Error:

```
{
  "success": false,
  "message": "Descripción del error",
  "errors": {
    "field": ["Error específico"]
  }
}
```

Listado con paginación:

```
{
  "success": true,
  "data": [...],
  "pagination": {
```

```
    "page": 1,
    "limit": 10,
    "total": 100,
    "total_pages": 10
  }
}
```

6. Códigos de Estado HTTP

Código	Nombre	Descripción
200	OK	Solicitud exitosa
201	Created	Recurso creado exitosamente
204	No Content	Solicitud exitosa sin contenido a retornar
400	Bad Request	Datos inválidos o faltantes
401	Unauthorized	Token JWT inválido o expirado
403	Forbidden	No tiene permisos para esta acción
404	Not Found	Recurso no encontrado
409	Conflict	Conflicto de datos (ej: email ya registrado)
422	Unprocessable Entity	Validación de datos falló
429	Too Many Requests	Rate limit excedido
500	Internal Server Error	Error interno del servidor
503	Service Unavailable	Servicio temporalmente no disponible

6.1. Cabeceras de Autenticación

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

El token JWT debe incluirse en todas las que requieran autenticación.

6.2. Rate Limiting

El sistema implementa rate limiting:

- **Autenticación**: 5 intentos por minuto
- **API general**: 60 requests por minuto
- **Escritura** (POST/PUT/DELETE)**: 30 requests por minuto

7. Códigos de Error Comunes

7.1. Errores de Autenticación

Código	Mensaje	Solución
auth-001	Token inválido	Verificar que el token esté bien formado
auth-002	Token expirado	Obtener nuevo token con /auth/login
auth-003	Usuario no encontrado	Verificar el email usado
auth-004	Contraseña incorrecta	Verificar la contraseña
auth-005	Email ya registrado	Usar otro email o iniciar sesión
auth-006	Cuenta no verificada	Verificar email antes de continuar

7.2. Errores de Torneos

Código	Mensaje	Solución
tourney-001	Tournament no encontrado	Verificar el ID del torneo
tourney-002	Tournament lleno	El tournament llegó a max_participants
tourney-003	Inscripción cerrada	El tournament ya comenzó o terminó
tourney-004	Saldo insuficiente	Recargar billetera antes de inscribirse
tourney-005	Ya inscrito	El usuario ya participa en el tournament
tourney-006	Session no encontrada	Verificar el ID de la sesión
tourney-007	Tiempo de selección vencido	Hacer picks antes del end_time
tourney-008	Número de selecciones inválido	Enviar el número exacto de selections

7.3. Errores de Eventos

Código	Mensaje	Solución
event-001	Evento no encontrado	Verificar el ID del evento
event-002	Evento ya liquidado	No se puede modificar un evento liquidado
event-003	Competidor no encontrado	Verificar el ID del competidor
event-004	Evento no asignado al tournament	Asignar el evento al tournament primero
event-005	Selección no encontrada	Verificar el ID de la selección
event-006	Evento ya iniciado	No se puede modificar un evento en curso

8. Apéndice: Ejemplos de Integración

8.1. Ejemplo: Integración Frontend - Obtener Torneos

```
// Obtener lista de torneos
const response = await fetch('http://localhost:8080/api/v1/
    tournaments');
const data = await response.c();

if (data.success) {
  const tournaments = data.data;
  tournaments.forEach(t => {
    console.log(`${t.name} - ${t.category.name}`);
  });
}
```

8.2. Ejemplo: Integración Frontend - Hacer Picks

```
// Autenticación
const loginRes = await fetch('http://localhost:8080/api/v1/auth/
    login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email: 'user@test.com', password: '
    pass123' })
});
const { data: { token } } = await loginRes.c();

// Hacer picks
const picksRes = await fetch('http://localhost:8080/api/v1/
    tournaments/1/sessions/picks', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`
  },
  body: JSON.stringify({
    session_id: 1,
    selection_ids: [1, 2, 3]
  })
});
const result = await picksRes.c();
console.log(result.message);
```

8.3. Ejemplo: Webhook de Liquidación (futuro)

```
// Cuando un evento se liquida, el sistema puede notificar
// a través de un webhook configurado en el tournament
POST https://mi-sistema.com/webhook/tournament
```

```
Content-Type: application/json

{
  "event_id": 1,
  "tournament_id": 1,
  "status": "settled",
  "results": [...],
  "winners": ["user_id_1", "user_id_2"]
}
```